

Parameter-efficient Fine-tuning

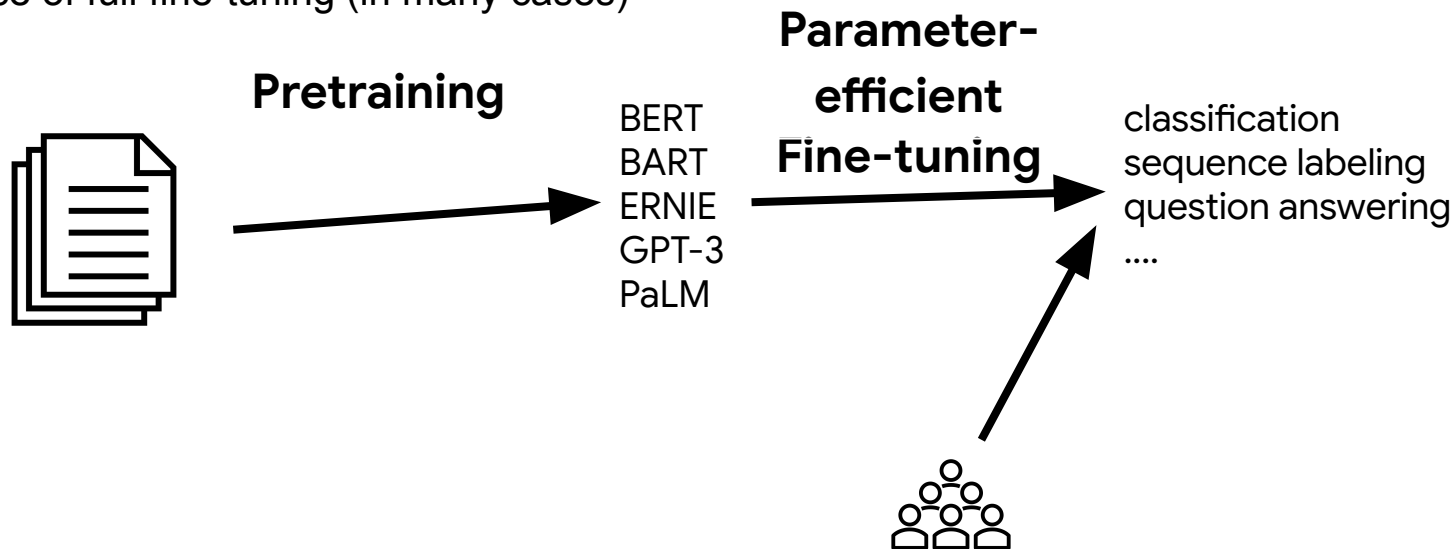
CS60216: Safety Fundamentals for Generative AI

Jan 27th, 2026

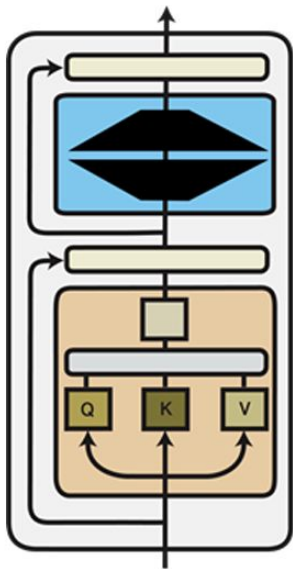
From Fine-tuning to Parameter-efficient Fine-tuning

Fine-tuning all parameters is impractical with large models

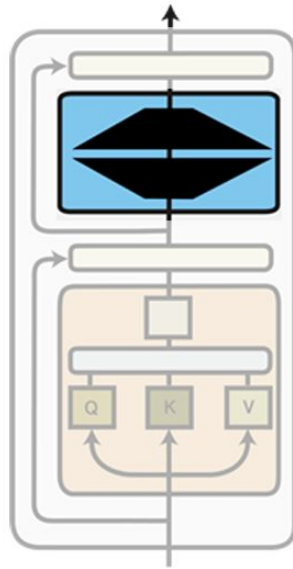
State-of-the-art models are massively overparameterized → Parameter-efficient finetuning matches performance of full fine-tuning (in many cases)



From fine-tuning to parameter efficient fine-tuning (PEFT)

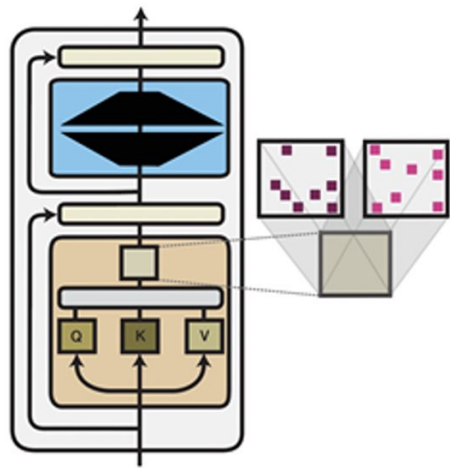


Full Fine-tuning
Update **all model parameters**

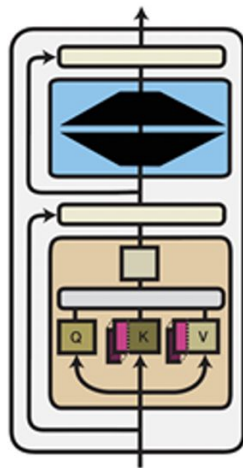


Parameter-efficient Fine-tuning
Update a **small subset** of model parameters

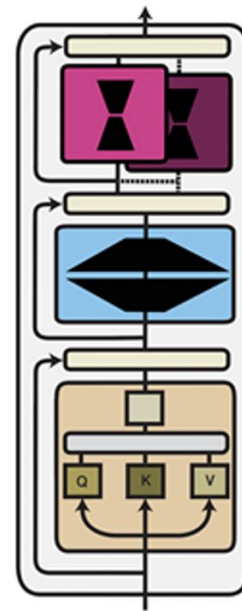
Different Perspectives to think about PEFT



Parameter



Input



Function

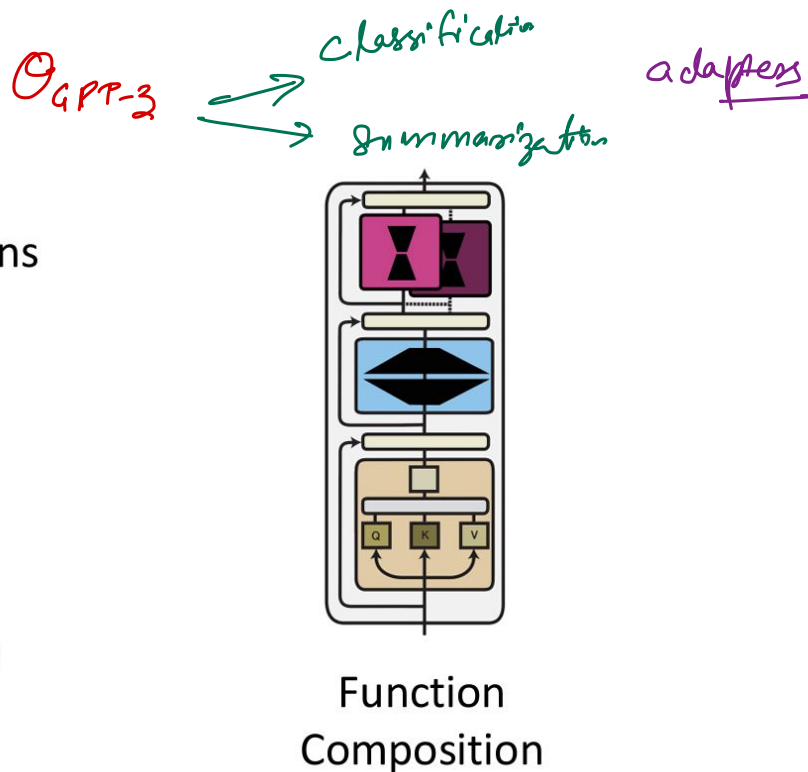
Functional perspective

Function composition augments a model's functions with **new task-specific functions**:

$$f'_i(\mathbf{x}) = f_{\theta_i}(\mathbf{x}) \odot f_{\phi_i}(\mathbf{x})$$

Most commonly used in multi-task learning where modules of different tasks are composed.

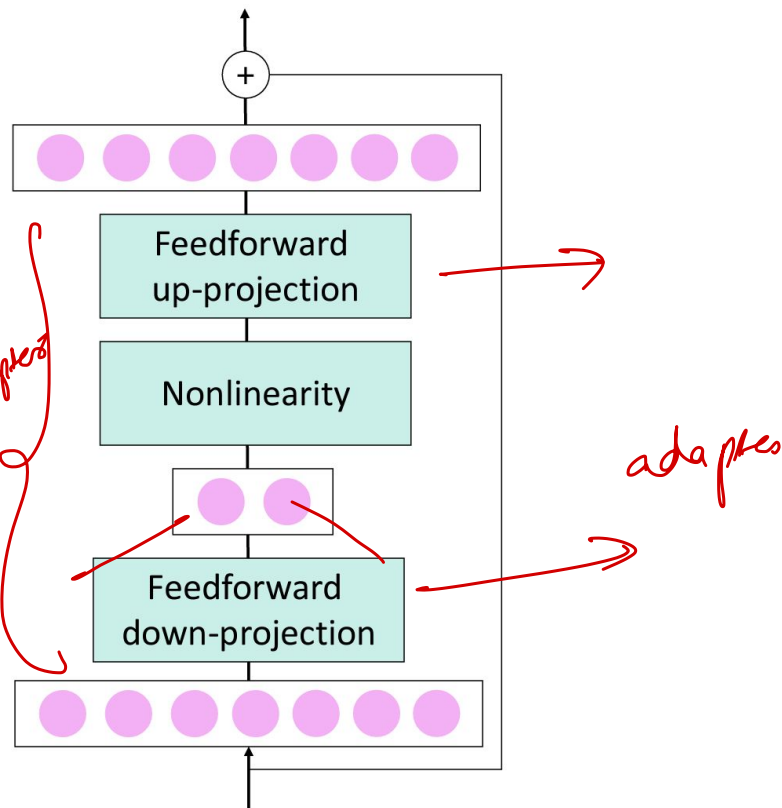
- Main purpose of functions f_{ϕ_i} added to a pre-trained model is to adapt it
→ Functions are also known as ‘adapters’



The Adapter Framework

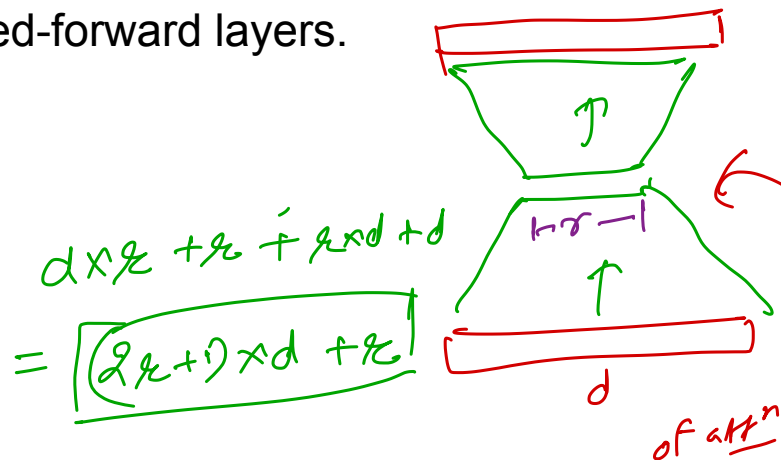
Adapter (Houlsby et al. 2019)

- Insert a new function f_ϕ between layers of a pre-trained model to **adapt** to a downstream task --- known as “adapters”
- An adapter in a Transformer layer consists of:
 - A feed-forward down-projection $W^D \in \mathbb{R}^{pk \times d}$
 - A feed-forward up-projection $W^U \in \mathbb{R}^{d \times k}$
- $f_\phi(x) = W^U(\sigma(W^D x))$

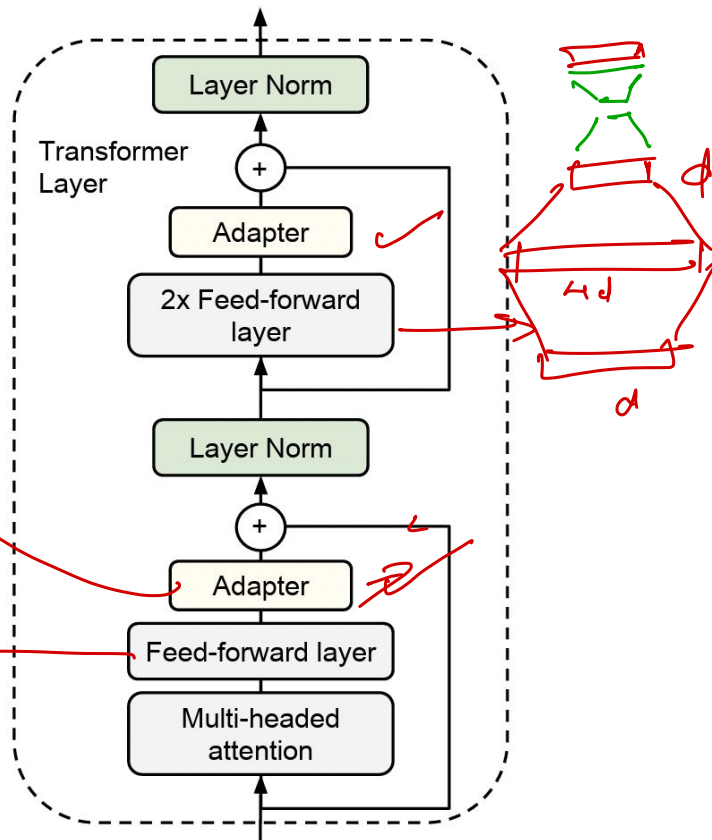


How many adapters per layer?

We add the adapter module twice to each Transformer layer: after the projection following multi-headed attention and after the two feed-forward layers.



Parameter-Efficient Transfer Learning for NLP:
<http://proceedings.mlr.press/v97/houlsby19a/houlsby19a.pdf>



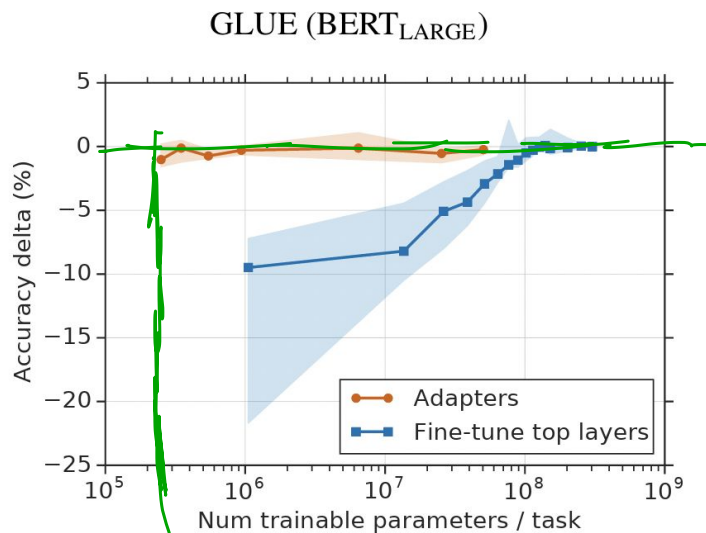
How many parameters?

Consider one adapter

- Feed-forward down-projection: $r \times d_{model} + r$
- Feed-forward up-projection: $d_{model} \times r + d_{model}$
- For L layers in the decoder, there would be $2L$ adapters
- Number of parameters: $\underline{2L} \times (\underline{2 \times d_{model} \times r} + \underline{d_{model}} + \underline{r})$

How is the performance?

	Total num params	Trained params / task	CoLA	SST	MRPC	STS-B	QQP	MNLI _m	MNLI _{mm}	QNLI	RTE	Total
BERT _{LARGE}	9.0×	100%	60.5	94.9	89.3	87.6	72.1	86.7	85.9	91.1	70.1	80.4
Adapters (8-256)	1.3×	3.6%	59.5	94.0	89.5	86.9	71.8	84.9	85.1	90.7	71.5	80.0
Adapters (64)	1.2×	2.1%	56.9	94.2	89.6	87.3	71.8	85.3	84.6	91.4	68.8	79.6



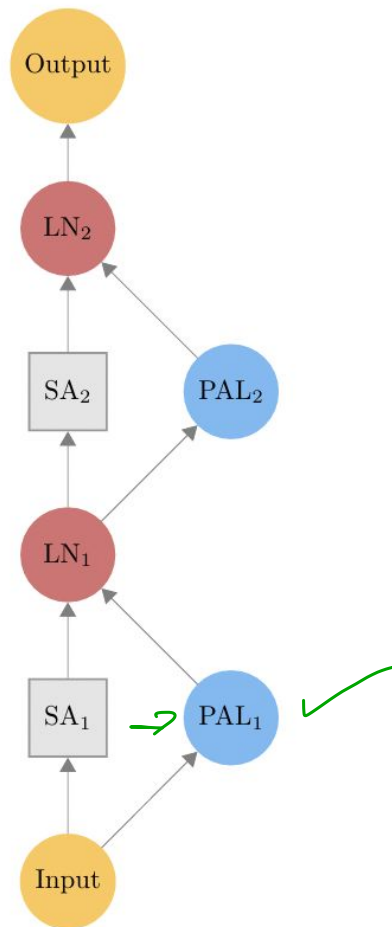
Adapter tuning achieves a comparable overall score (80.0) to full fine-tuning (80.4) using only 1.3x parameters in total, compared to 9x. Fixing the adapter size to 64 leads to a slightly decreased overall score of 79.6 and a slightly smaller model.

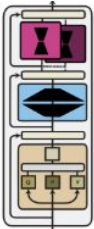
housby19a

Adapters in parallel

$$\text{TS}(\mathbf{h}) = V^D g(V^E \mathbf{h})$$

$$\mathbf{h}^{l+1} = \text{LN}(\mathbf{h}^l + \text{SA}(\mathbf{h}^l) + \text{TS}(\mathbf{h}^l))$$



		Parameter efficiency	Training efficiency	Inference efficiency	Performance
Function Composition		Adapters depend on the hidden size	Does not require gradients of frozen params	New functions increase # of operations	Match or outperform standard fine-tuning